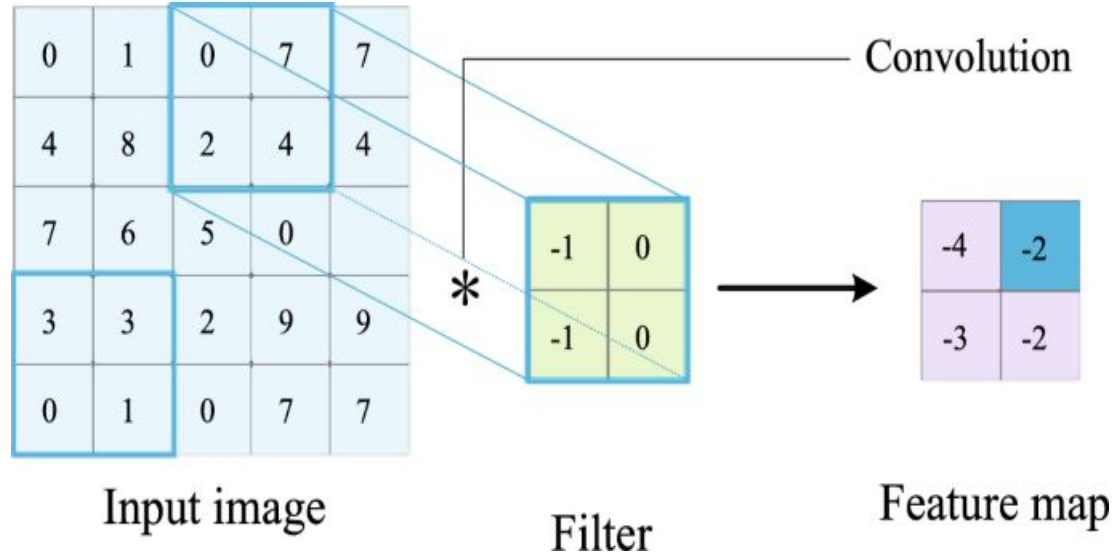


Convolution in NN



You are given the following 4×4 input image matrix:

$$I = \begin{bmatrix} 1 & 2 & 3 & 0 \\ 4 & 5 & 6 & 1 \\ 7 & 8 & 9 & 2 \\ 1 & 3 & 2 & 0 \end{bmatrix}$$

and the following 3×3 filter (kernel):

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Assume:

- **Stride = 1**
- **No padding**

Q.1. Perform the convolution operation and compute the **output feature map**, showing the calculation for each step.

Q.2. What will be:

- The size of the output feature map?
- The value at position (1,1) after convolution?

Convolution Calculation (Stride=1, No Padding)

Input:

```
[ [1,2,3,0],  
  [4,5,6,1],  
  [7,8,9,2],  
  [1,3,2,0] ]
```

Kernel:

```
[ [1,0,-1],  
  [1,0,-1],  
  [1,0,-1] ]
```

Output size: $(4-3+1) \times (4-3+1) = 2 \times 2$

--- Output(0,0) ---

```
[[1,2,3],  
 [4,5,6],  
 [7,8,9]]
```

Row1: $1-3 = -2$

Row2: $4-6 = -2$

Row3: $7-9 = -2$

Sum = -6

--- Output(0,1) ---

```
[[2,3,0],  
 [5,6,1],  
 [8,9,2]]
```

Row1: $2-0 = 2$

Row2: $5-1 = 4$

Row3: $8-2 = 6$

Sum = 12

--- Output(1,0) ---

```
[[4,5,6],  
 [7,8,9],  
 [1,3,2]]
```

Row1: $4-6 = -2$

Row2: $7-9 = -2$

Row3: $1-2 = -1$

Sum = -5

--- Output(1,1) ---

```
[[5,6,1],  
 [8,9,2],  
 [3,2,0]]
```

Row1: $5-1 = 4$

Row2: $8-2 = 6$

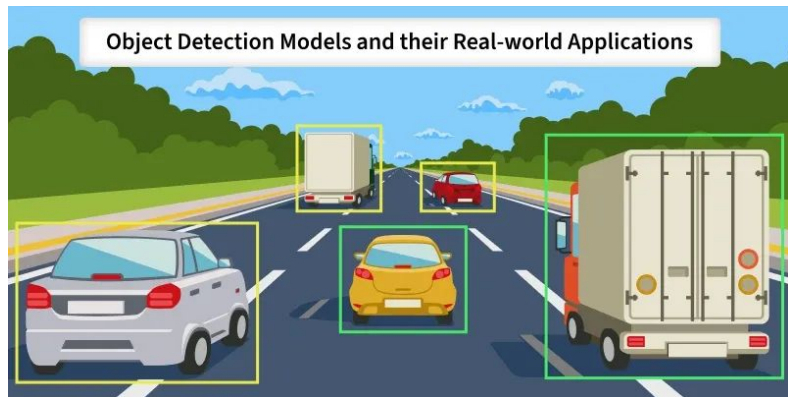
Row3: $3-0 = 3$

Sum = 13

Final output:

```
[[ -6, 12],  
 [ -5, 13]]
```

Object Detection and Image Segmentation

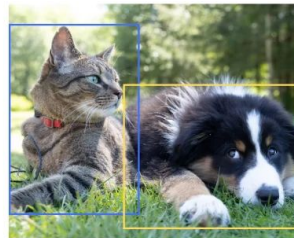


Is this a cat?



Image Classification

What is there in the image and where?



Object Detection

Which pixels belong to which object



Image Segmentation

Image Classification: A core task in Computer Vision



[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}



cat

Computer Vision Tasks

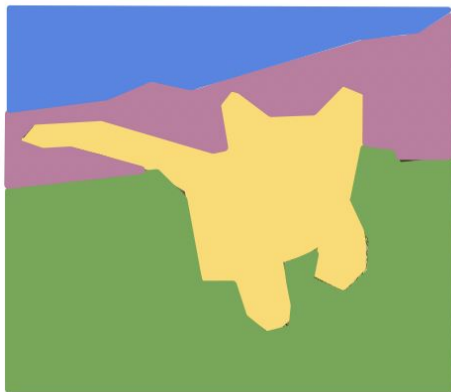
Classification



CAT

No spatial extent

Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

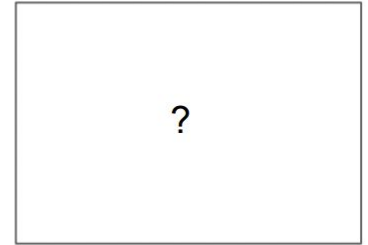
[This image](#) is [CC0 public domain](#)

Semantic Segmentation: The Problem



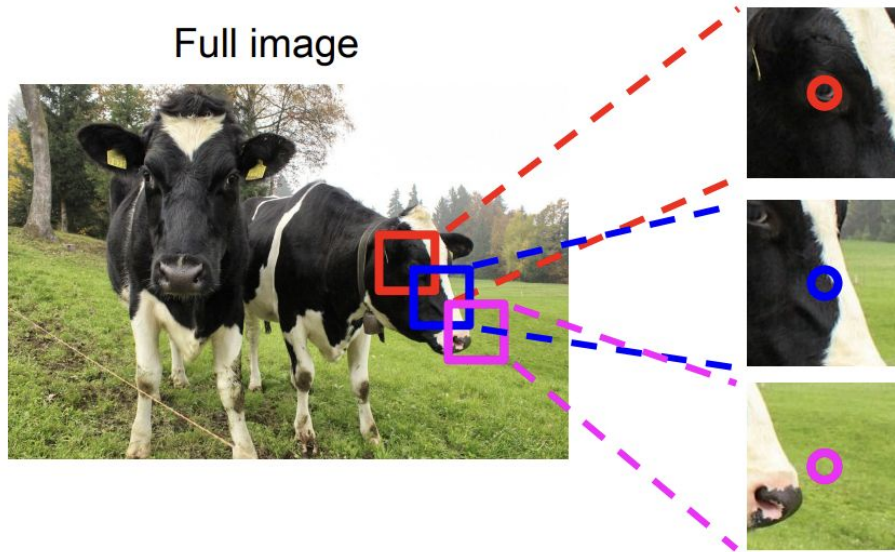
GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



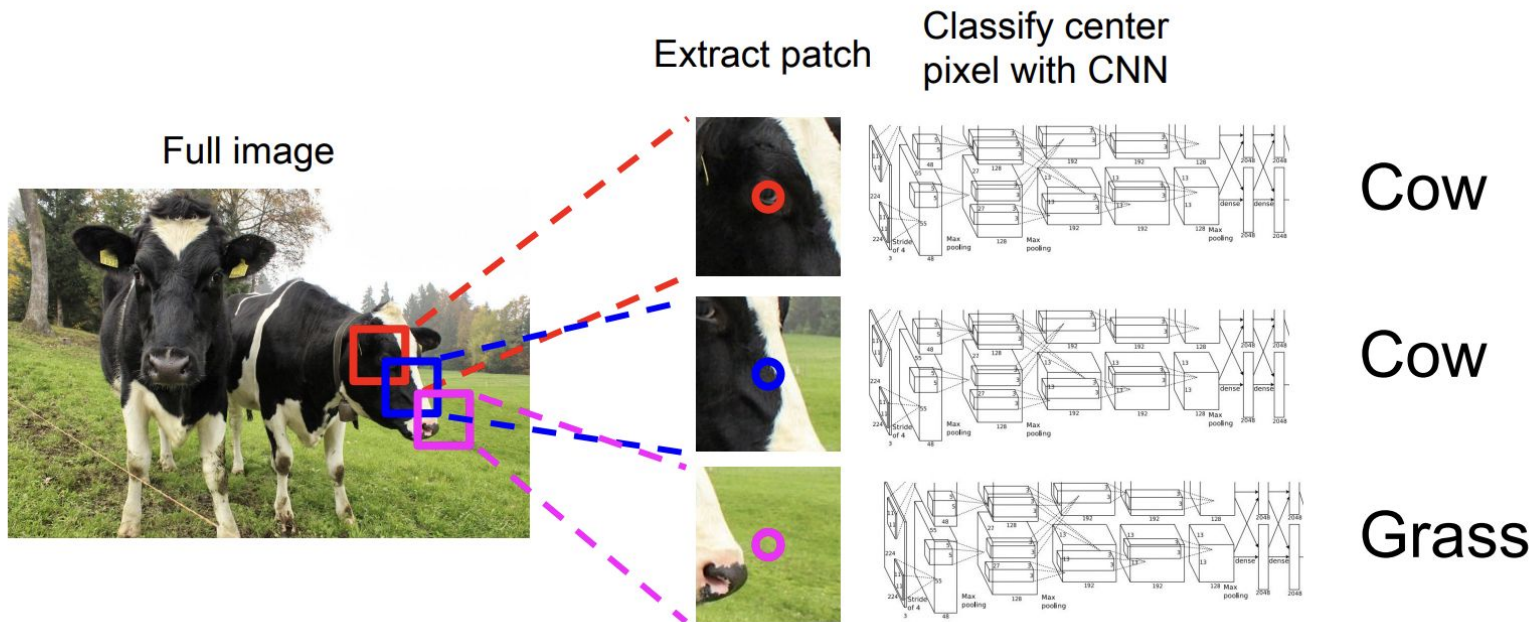
At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window



Q: how do we model this?

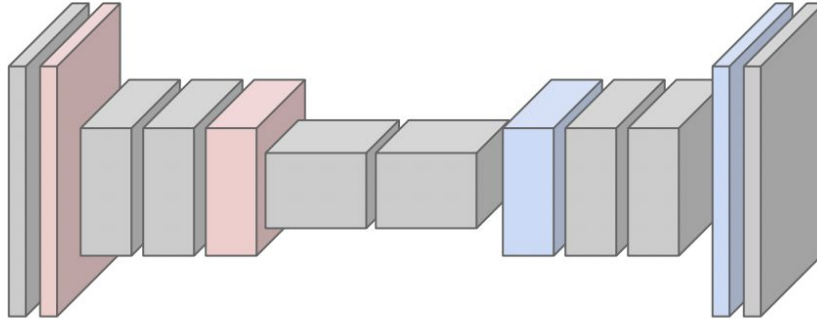
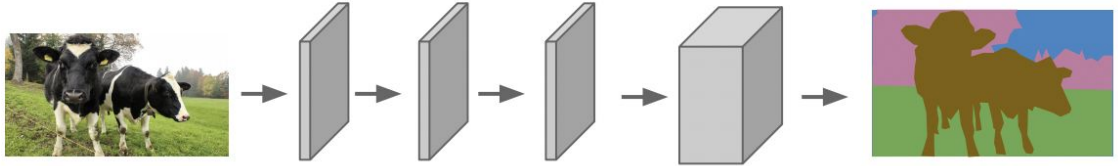
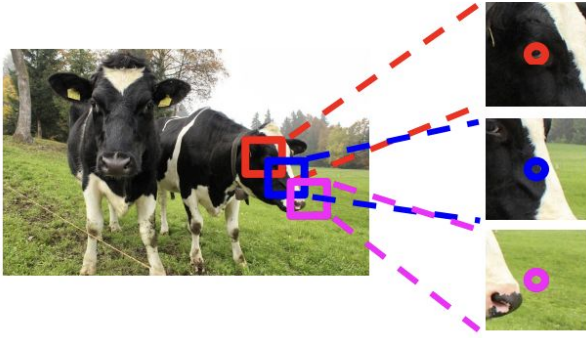
Semantic Segmentation Idea: Sliding Window



Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

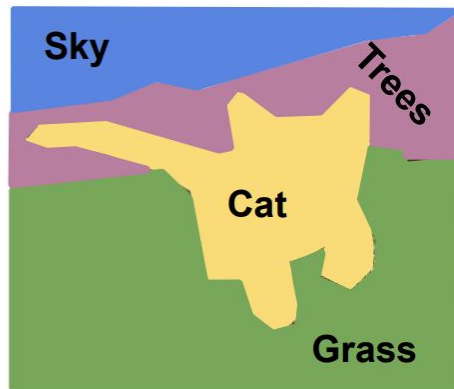
Semantic Segmentation: Summary



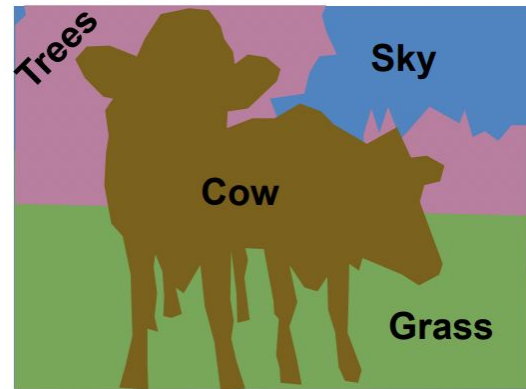
Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels

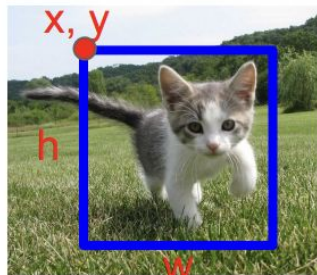


[This image is CC0 public domain](#)

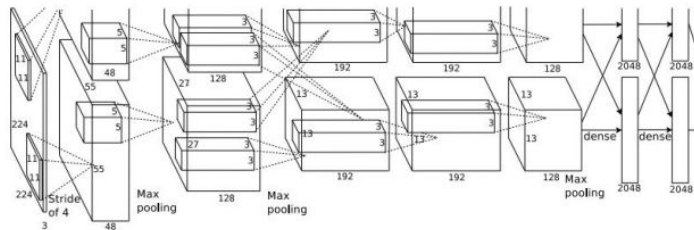


Object Detection: Single Object

(Classification + Localization)



[This image is CC0 public domain](#)



**Fully
Connected:**
4096 to 1000

Class Scores

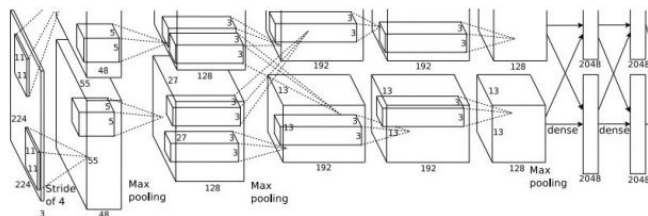
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Vector:
4096

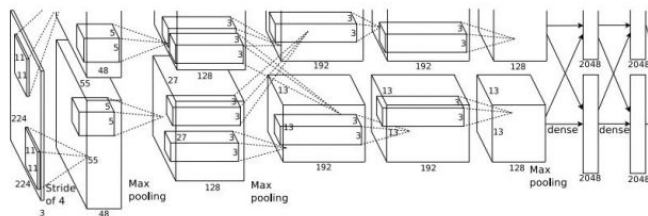
**Fully
Connected:**
4096 to 4

**Box
Coordinates**
(x, y, w, h)

Object Detection: Multiple Objects



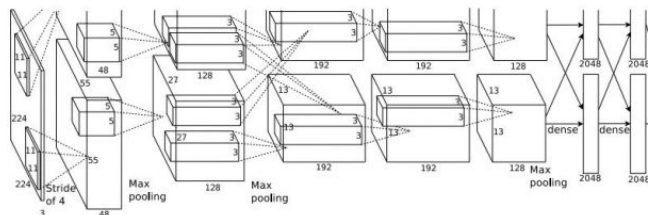
CAT: (x, y, w, h)



DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)



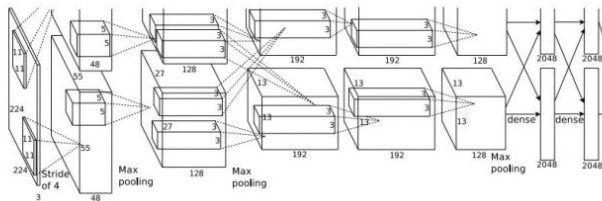
DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

....

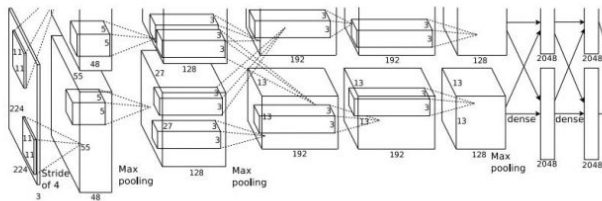
Object Detection: Multiple Objects

Each image needs a different number of outputs!



CAT: (x, y, w, h)

4 numbers

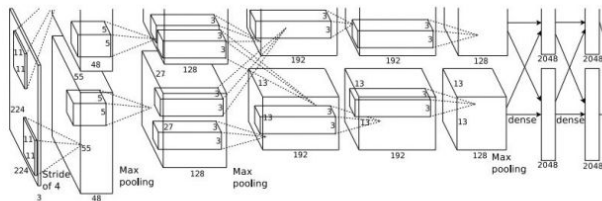


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

12 numbers



DUCK: (x, y, w, h)

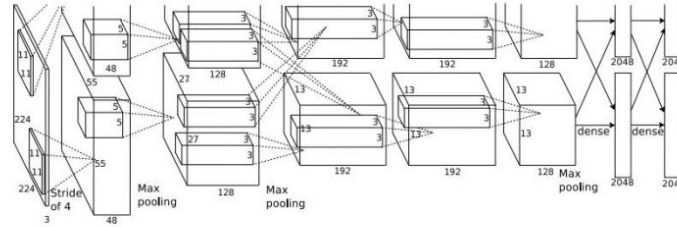
DUCK: (x, y, w, h)

Many numbers!

....

Object Detection: Multiple Objects

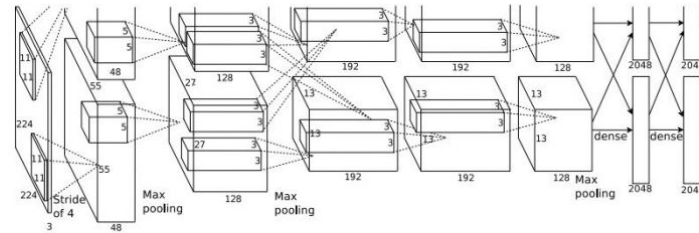
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



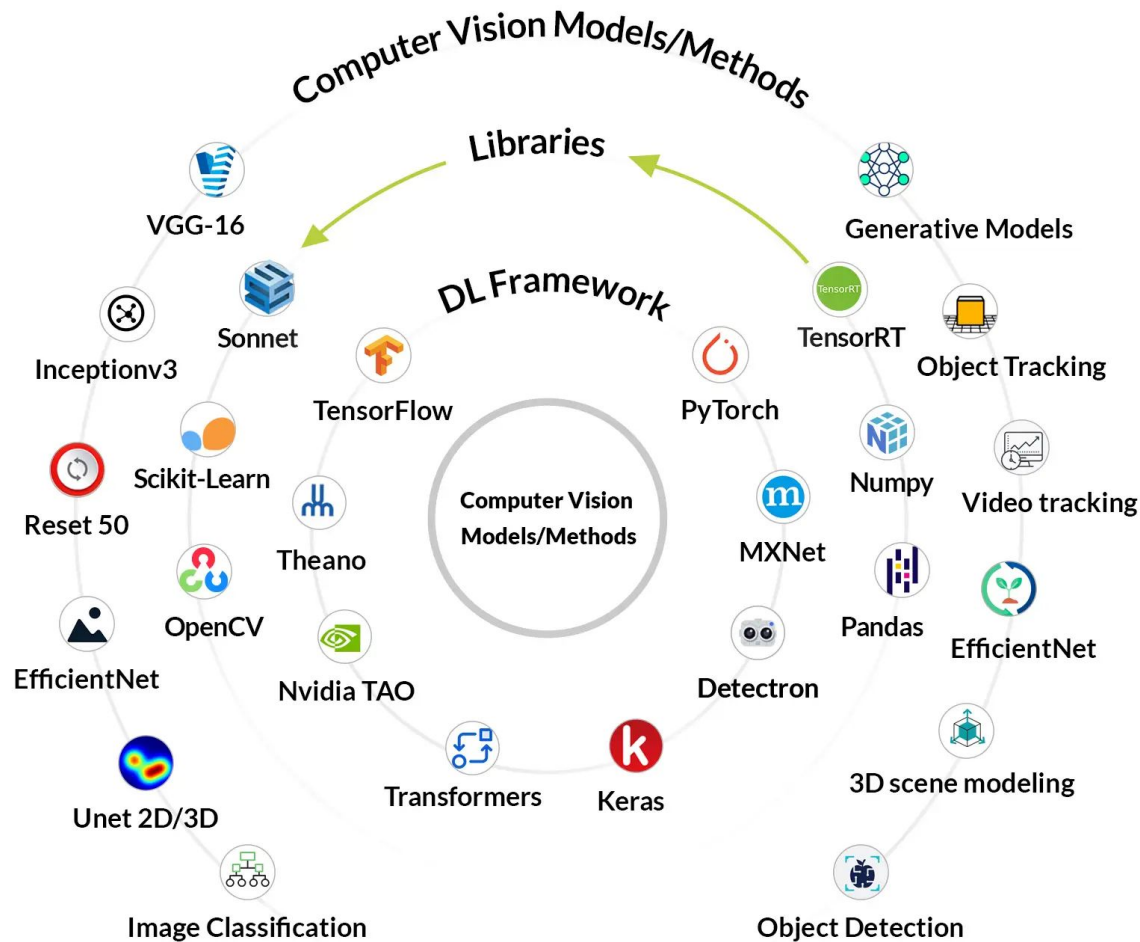
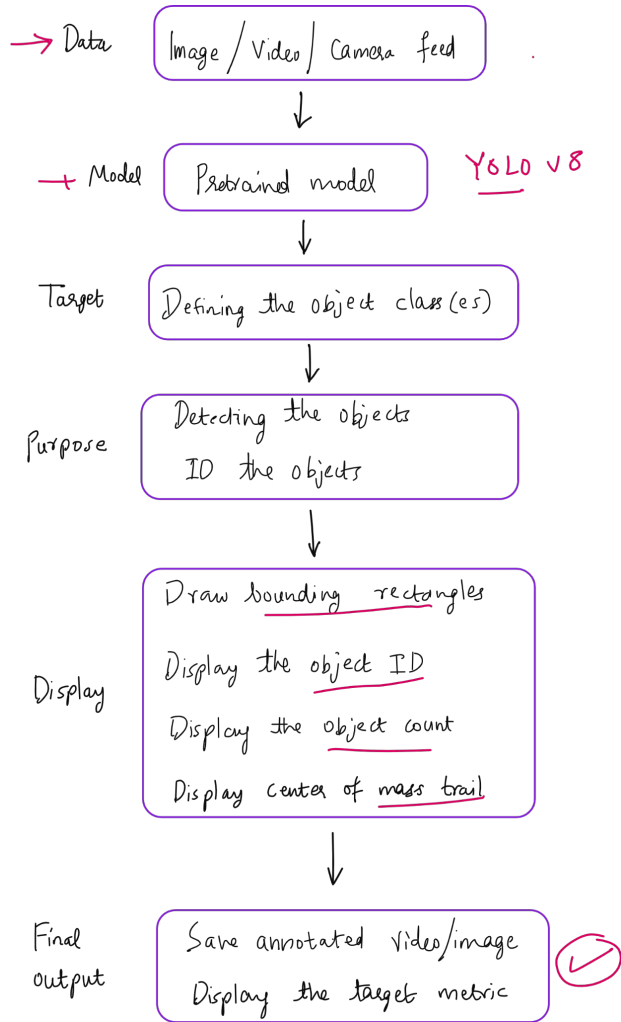
Dog? NO
Cat? NO
Background? YES

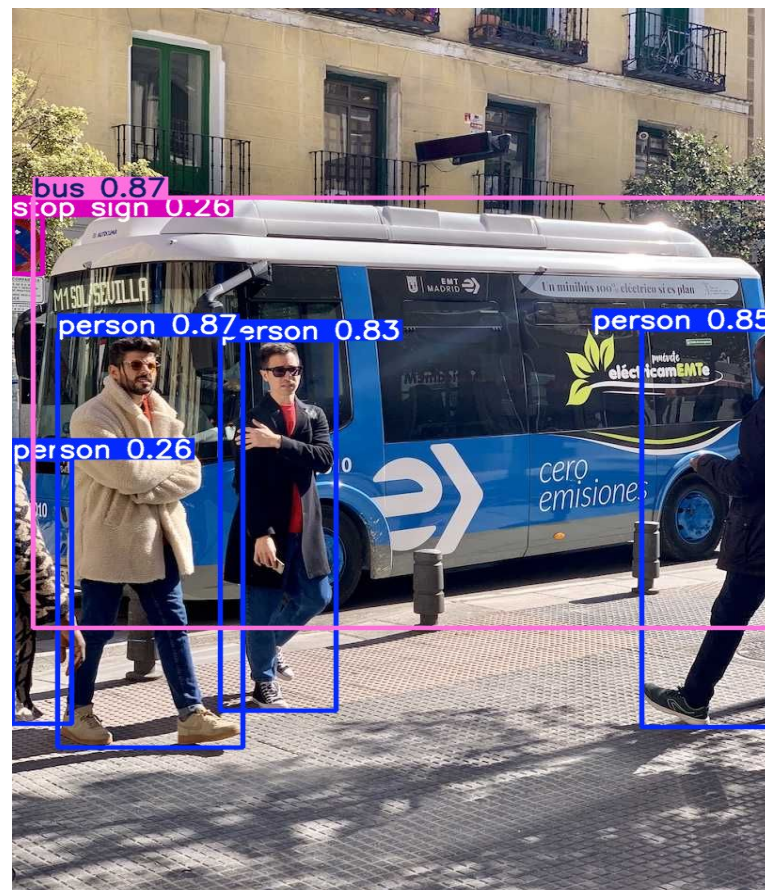
Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? YES
Cat? NO
Background? NO





- Installs the **Ultralytics YOLO package**
- This package provides pre-trained YOLOv8 model to run detection easily.
- Imports the YOLO model class.
- Imports OpenCV for image handling.
- `cv2_imshow()` is used for showing images
- Loads a pre-trained YOLOv8 model ([yolov8n.pt](#)).
- **n** stands for **nano version** (small, fast, lightweight).
- This model is trained to detect 80 common object categories
- Downloads a sample image from the internet.
- Saves it as `sample.jpg`.
- Reads the image into memory using OpenCV.
- Displays the **original image before object detection**.

```
!pip install ultralytics

Show hidden output

from ultralytics import YOLO
import cv2
from google.colab.patches import cv2_imshow

Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com

model = YOLO("yolov8n.pt") # n = nano version

Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt to 'yolov8n.pt': 100%

!wget https://ultralytics.com/images/bus.jpg -O sample.jpg

Show hidden output

# Read image with OpenCV
image = cv2.imread("sample.jpg")

print("Original Image:")
cv2_imshow(image)
```


- Runs YOLO on the input image.
- `results` contains detection information (bounding boxes, labels, scores).
- `show=False` means do not automatically open a new window for the result.
- `results[0]` refers to the first prediction result.
- `.plot()` draws the bounding boxes and labels on the image.
-

```
results = model.predict(source="sample.jpg", show=False)
```

```
image 1/1 /content/sample.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 409.7ms  
Speed: 10.9ms preprocess, 409.7ms inference, 37.0ms postprocess per image at shape (1, 3, 640, 480)
```

```
▶ # Plot the result image  
results[0].plot() # creates an image with bounding boxes  
cv2.imshow(results[0].plot())
```